



Новый навык за несколько недель

[Войти](#)**honest_niceman**

24 минуты назад

TypeScript 7.0 — самое радикальное обновление за 10 лет

Простой

12 мин

776

Блог компании Haulmont, Программирование*, JavaScript*, TypeScript*

[Обзор](#)[Перевод](#)

Автор оригинала: Daniel Rosenwasser

С самого начала TypeScript обещал обеспечить масштабируемость JavaScript. Строгая типизация и богатый инструментарий позволили разрабатывать нетривиальные, высококачественные приложения под любые платформы.

В прошлом году наша команда анонсировала следующий шаг в масштабировании TypeScript: нативный порт на Go, который использует все возможности современного железа. Порт сделан максимально точно: новый код воспроизводит структуру и логику оригинальной кодовой базы, чтобы результаты двух компиляторов совпадали.

Ключевое отличие — TypeScript 7 использует скорость нативного кода, разделяемую память при многопоточности и ряд новых оптимизаций. В итоге полная сборка ускоряется в 8–12 раз.

Установка

```
npm install -D typescript
```

Объяснить с SourceCraft

Эта команда установит новый исполняемый файл `tsc` в рабочее пространство (запускается через `npm install -g tsc`).

Важная часть опыта TypeScript — поддержка в редакторах. Любой современный редактор с поддержкой Language Server Protocol (LSP) легко подхватит TypeScript 7 с его новой скоростью и многопоточностью. OpenIDE, VS Code, Visual Studio, WebStorm и другие редакторы работают без дополнительных настроек.

- Для VS Code есть [отдельное расширение TypeScript 7](#), доступное уже сейчас. Visual Studio включит TypeScript 7 автоматически на основе настроек рабочего пространства.
- В OpenIDE отдельный плагин [Frontend/Web](#) также предоставляет поддержку TypeScript всех актуальных версий.

Что даёт ускоренный TypeScript?

Быстрый TypeScript звучит хорошо на бумаге, но что это означает на практике?

Типичный день разработчика: открыть IDE, открыть TypeScript-файл, запустить «найти все ссылки» по всему проекту. Потом — редактировать, ожидать автодополнений, видеть красные подчёркивания в реальном времени. Когда пришло время собирать проект (или это делает AI-агент) — запустить `tsc`, проверить вывод на ошибки, запустить сгенерированный код.

TypeScript 7 ускоряет каждый этап:

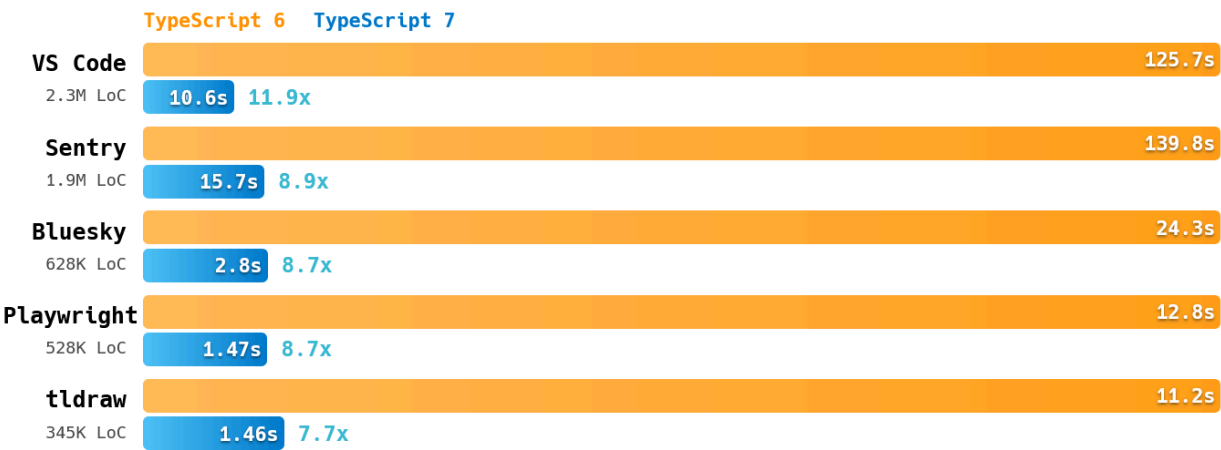
- Загрузка проекта в редакторе становится практически мгновенной.
- «Найти все ссылки», автодополнение и диагностика — работают в долю прежнего времени.
- `tsc` в режиме `--watch` — цикл обратной связи становится значительно короче.

Вот результаты на реальных крупных open-source проектах:

Проект	TypeScript 6	TypeScript 7	Ускорение
vscode	125,7 с	10,6 с	11,9×
sentry	139,8 с	15,7 с	8,9×
bluesky	24,3 с	2,8 с	8,7×

playwright	12,8 c	1,47 c	8,7×
tldraw	11,2 c	1,46 c	7,7×

Compile Times Between TypeScript 6 & 7

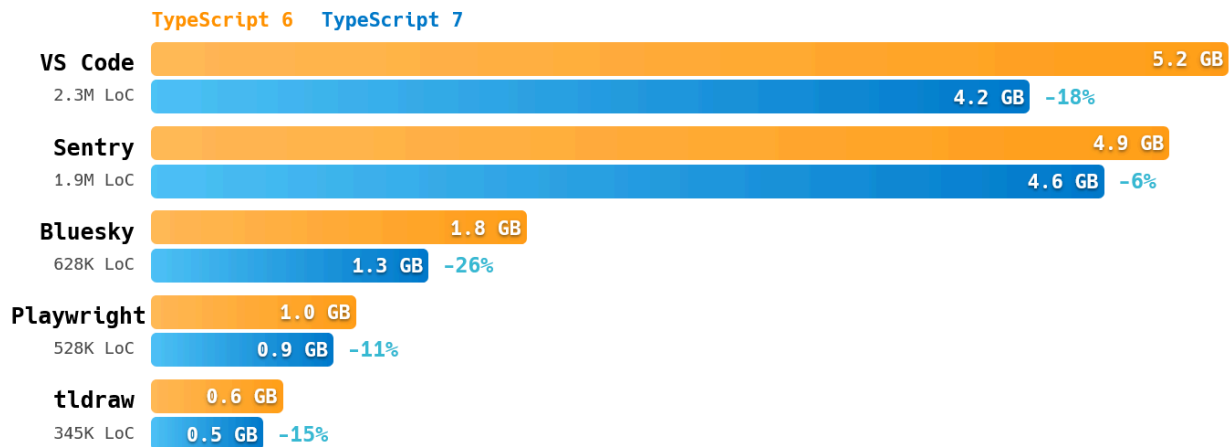


Время сборки проектов в TypeScript 6 и 7: ускорение от 7,7× до 11,9×

TypeScript 7 также потребляет меньше памяти:

Проект	TypeScript 6	TypeScript 7	Изменение
vscode	5,2 ГБ	4,2 ГБ	-18%
sentry	4,9 ГБ	4,6 ГБ	-6%
bluesky	1,8 ГБ	1,3 ГБ	-26%
playwright	1,0 ГБ	0,9 ГБ	-11%
tldraw	0,6 ГБ	0,5 ГБ	-15%

Aggregate Memory Usage in TypeScript 6 & 7



Снижение потребления памяти в TypeScript 7 по сравнению с TypeScript 6: от -6% до -26%

И это ещё не всё. На одном и том же компьютере открытие файла с ошибкой в кодовой базе VS Code раньше занимало около 17,5 секунды с момента открытия редактора до первой видимой ошибки. С TypeScript 7 это занимает менее 1,3 секунды — более чем в 13 раз быстрее.

Проверен в бою и готов к продакшену

В проекте TypeScript накоплены десятки тысяч тестов, которые запускаются на каждый коммит в ветке `main` на протяжении более десяти лет. Они обеспечивают стабильность и надёжность каждого релиза.

Но TypeScript 7 — не обычный релиз. Помимо тестового набора, мы использовали множество дополнительных ресурсов, чтобы убедиться в его готовности к продакшену.

За последний год мы работали со многими крупными командами внутри компании и снаружи, тестируя TypeScript 7 на реальных кодовых базах. Результаты оказались весьма позитивными: целые компании сообщали, что TypeScript 7 стабилен, быстр и легко внедряется. Например, команда VS Code недавно рассказала о своём опыте работы с предварительными версиями TypeScript 7 в рамках ускорения цикла разработки. Мы также работали с командами Microsoft: Loop, Office, PowerBI, Teams и Xbox — чтобы убедиться, что TypeScript готов к крупнейшим кодовым базам. Bloomberg, Canva, Figma, Google, Lattice, Linear, Miro, Notion, Sentry, Slack, Vanta, Vercel, VoidZero и другие компании помогли тестировать TypeScript 7 и делились обратной связью.

Кроме того, мы перестроили большую часть тестовой инфраструктуры для работы с TypeScript 7: автоматизированное и запускаемое по запросу тестирование TypeScript-и

JavaScript-проектов на GitHub для обнаружения регрессий в компиляторе и языковом сервере.

Благодаря явной обратной связи, автоматическим отчётам о сбоях и агрессивному тестированию качество заметно выросло. Данные показывают, что новый языковой сервер TypeScript 7.0 сократил количество неудачных команд более чем на 80%, а число сбоев сервера — более чем на 60% по сравнению с TypeScript 6.0.

Отзывы команд о реальных результатах:

- **Slack:** TypeScript 7 устранил 40% времени ожидания в очереди слияний и сократил время проверки типов в CI с ~7,5 минут до 1,25 минуты. Локальная разработка в редакторе раньше была практически «непригодной для использования» из-за времени загрузки языкового сервера — инженеры обычно перекладывали полную проверку типов на CI. TypeScript 7 загружает ту же кодовую базу за несколько секунд, и локальная проверка типов снова стала реальной.
- **Vanta:** сборки резко ускорились — до 9× на одном из крупнейших проектов.
- **Microsoft News Services:** внедрение TypeScript 7 сэкономило команде 400 часов в месяц, которые раньше уходили на ожидание CI-сборок.
- **PowerBI:** инженеры называли TypeScript 7 в редакторе «жизнеспасающим» для работы с их кодовой базой и приняли этот опыт как стандарт ещё до того, как TypeScript 7 поддержал переименование в VS Code.
- **Loop:** предыдущий опыт в редакторе описывался как «непригодный для использования» в их масштабе, тогда как TypeScript 7 оказался «потрясающим».
- **Canva:** языковой сервер TypeScript 7 показал разительное ускорение — время до появления первой ошибки в редакторе сократилось примерно с 58 секунд до 4,8 секунды.

Параллельная работа с TypeScript 6.0

TypeScript 7.0 не поставляется с API. Мы ожидаем, что TypeScript 7.1 выйдет с новым (и другим) API, но пока что приоритетом для нас стало обеспечение параллельной работы TypeScript с TypeScript 6.0 для инструментов, которым по-прежнему нужен программный доступ к компилятору (например, typescript-eslint).

В рамках процесса перехода с 6.0 на 7.0 мы опубликовали новый пакет совместимости — `@typescript/typescript6`. Он предоставляет исполняемый файл с именем `tsc6`, так что при необходимости можно установить TypeScript 7.0 (со своим бинарником `tsc`) рядом без конфликтов имён. Новый пакет также реэкспортирует API TypeScript 6.0.

Поскольку некоторые инструменты вроде `typescript-eslint` ожидают импорт непосредственно из `typescript` через `peer`-зависимости, рекомендуем использовать `npm`-псевдонимы:

```
npm install -D typescript@npm:@typescript/typescript6
```

Объяснить с 

или изменить `package.json` :

```
{
  "devDependencies": {
    "typescript": "npm:@typescript/typescript6@^6.0.2"
  }
}
```

Объяснить с 

Это оставит вам только исполняемый файл `tsc6` . Чтобы добавить `tsc` от TypeScript 7.0, добавьте ещё один псевдоним:

```
{
  "devDependencies": {
    "@typescript/native": "npm:typescript@^7.0.2",
    "typescript": "npm:@typescript/typescript6@^6.0.2"
  }
}
```

Объяснить с 

Ночные сборки и `@typescript/native-preview`

До этого момента большинство разработчиков устанавливали TypeScript 7 через пакет `@typescript/native-preview` . Этот пакет выпускал ночные сборки новой кодовой базы — он хорошо служил сообществу с более чем 8,5 миллиона скачиваний в неделю!

Однако в дальнейшем ночные сборки возобновятся под стандартным пакетом `typescript` с тегом `next` :

```
npm install -D typescript@next
```

Объяснить с 

Управление масштабированием: параллелизация и флаги

TypeScript 7.0 теперь выполняет многие шаги параллельно: разбор (парсинг), проверку типов и генерацию кода. Некоторые из них, например парсинг и генерация, выполняются практически независимо для каждого файла — параллелизация автоматически хорошо масштабируется на больших кодовых базах с минимальными накладными расходами. Но не каждый шаг в сборке TypeScript легко распараллелить.

TypeScript 7 вводит экспериментальные флаги `--checkers` и `--builders` для тонкой настройки параллелизации таких шагов, как проверка типов и сборка ссылок на проекты. Также добавлен флаг `--singleThreaded` для полного отключения параллелизации — полезен при отладке или в средах с ограниченными ресурсами.

Параллелизация проверки типов

Проверка типов сложнее из-за зависимостей между файлами. Большинство файлов зависят от одних и тех же типов из зависимостей и глобальной области видимости, поэтому запускать независимые проверщики типов было бы расточительно — и по вычислениям, и по памяти. С другой стороны, порядок файлов иногда важен, поэтому проверка с нуля должна обходить их в одном и том же порядке для одинаковых результатов.

TypeScript 7.0 создаёт фиксированное число воркеров-проверщиков типов со своим видением мира. Они могут дублировать часть общей работы, но при одинаковых входных файлах всегда разделят их идентично и дадут те же результаты.

По умолчанию используется 4 воркера проверки типов, но это настраивается флагом `--checkers`. Увеличение числа на крупных кодовых базах с большим числом CPU-ядер может дополнительно ускорить сборку, но обычно ценой роста памяти.

Для сравнения: вот результаты с `--checkers 8` на том же оборудовании (в таблице выше — `--checkers 4` по умолчанию):

Проект	TypeScript 6	TypeScript 7 (<code>--checkers 8</code>)	Ускорение
vscode	125,7 с	7,51 с	16,7×
sentry	139,8 с	12,08 с	11,6×
bluesky	24,3 с	2,01 с	12,1×
playwright	12,8 с	1,16 с	11×
tldraw	11,2 с	1,06 с	10,6×

На машинах с меньшим числом ядер и памяти (например, в CI) имеет смысл уменьшить это значение. Можно указать `--checkers 1`, фактически переведя проверку типов в однопоточный режим и устранив дублирование работы.

В редких случаях разное число `--checkers` может давать разные результаты из-за зависимостей от порядка. Фиксирование числа чекеров во всех сборочных окружениях помогает гарантировать одинаковые результаты.

Параллелизация сборки ссылок на проекты

TypeScript 7.0 умеет параллелизировать сборку не только внутри одного проекта, но и сразу несколько проектов. Это настраивается флагом `--builders`, который управляет числом параллельных строителей ссылок на проекты при запуске с `--build`. Особенно полезно для монорепозитория со множеством проектов.

Как и `--checkers`, увеличение числа строителей может ускорить сборку, но потребует больше памяти. Эффекты умножаются, поэтому важно найти баланс: например, `--checkers 4 --builders 4` допускает до 16 одновременных проверщиков типов — это может быть избыточным.

В отличие от `--checkers`, разное число строителей не должно давать разные результаты; однако сборка ссылок на проекты принципиально ограничена графом зависимостей проектов (за исключением случаев использования `--isolatedDeclarations`).

Однопоточный режим

Иногда полезно принудительно ограничить компилятор одним потоком — для отладки, сравнения производительности TypeScript 6 и 7, оркестрации параллельных сборок извне или работы в средах с очень ограниченными ресурсами. Флаг `--singleThreaded` ограничивает не только число воркеров проверки типов до 1, но и гарантирует, что парсинг и генерация кода выполняются в одном потоке.

Улучшенный режим `--watch`

TypeScript 7 поставляется с полностью переработанным режимом `--watch`, основанным на Parcel-вотчере — файловом наблюдателе Parcel Bundler, который обеспечивает эффективное и стабильное кроссплатформенное отслеживание изменений файлов.

При портировании логики файлового наблюдения мы столкнулись с несколькими проблемами кроссплатформенного file watching в Go: стандартная библиотека не предоставляет встроенного API, а сторонние библиотеки имели проблемы со стабильностью, производительностью или интеграцией с инструментами сборки.

Решения на основе периодического опроса работали везде, но были вычислительно дорогими — особенно на больших проектах с множеством зависимостей в `node_modules`.

Visual Studio Code многие годы полагается на `@parcel/watcher`. Мы портировали его на Go с минимальными ассемблерными вставками, чтобы не добавлять зависимость от тулчейна C++.

Результат превзошёл ожидания: что начиналось как прямой перевод с C++ на Go, превратилось в идиоматичный Go-код, который по-прежнему проходит портированный тестовый набор. Мы видим значительное улучшение ресурсопотребления в режиме `--watch` на всех платформах.

Отдельная благодарность Devon Govett, чья работа над Parcel принесла огромную пользу и VS Code, и TypeScript.

Обновления с версии 5.x и новые поведения из 6.0

TypeScript 7.0 совместим с TypeScript 6.0 по проверке типов и поведению командной строки. Практически любой TypeScript-код, который чисто компилируется в TypeScript 6.0 (с флагом `stableTypeOrdering` и без `ignoreDeprecations`), должен компилироваться идентично в TypeScript 7.0.

TypeScript 7.0 принимает новые умолчания 6.0 и генерирует жёсткие ошибки при использовании флагов и конструкций, помеченных как устаревшие в TypeScript 6.0. Мы рекомендуем сначала перейти на TypeScript 6.0, чтобы облегчить переход на 7.0. Подробнее об устаревших возможностях — в [блог-посте о выходе TypeScript 6.0](#).

Изменения умолчаний конфигурации

- `strict` по умолчанию `true`.
- `module` по умолчанию `esnext`.
- `target` по умолчанию — текущая стабильная версия ECMAScript, предшествующая `esnext`.
- `noUncheckedSideEffectImports` по умолчанию `true`.
- `libReplacement` по умолчанию `false`.
- `stableTypeOrdering` по умолчанию `true` и не может быть отключён.
- `rootDir` теперь по умолчанию `./`, внутренние исходные директории нужно указывать явно.
- `types` теперь по умолчанию `[]`; старое поведение возвращается через `["*"]`.

Изменения `rootDir` и `types` могут быть самыми «неожиданными», но легко устраняются.

Для `rootDir` (если `tsconfig.json` лежит вне директории вроде `src`):

```
{
  "compilerOptions": {
    "rootDir": "./src"
  },
  "include": ["./src"]
}
```

Объяснить с 

Для `types` (нужно явно перечислить используемые `@types` -пакеты):

```
{
  "compilerOptions": {
    "types": ["node", "jest"]
  }
}
```

Объяснить с 

Устаревшие возможности, превратившиеся в жёсткие ошибки

- `target: es5` — больше не поддерживается.
- `downlevelIteration` — больше не поддерживается.
- `moduleResolution: node/node10` — не поддерживается; рекомендуется `nodenext` или `bundler`.
- `module: amd, umd, systemjs, none` — не поддерживаются; рекомендуется `esnext` или `preserve` совместно с бандлерами.
- `baseUrl` — не поддерживается; обновите `paths` на пути относительно корня проекта.
- `moduleResolution: classic` — не поддерживается; рекомендуется `bundler` или `nodenext`.
- `esModuleInterop` и `allowSyntheticDefaultImports` нельзя установить в `false`.
- `alwaysStrict` считается `true` и не может быть выключен.

- Ключевое слово `module` нельзя использовать в объявлениях пространств имён.
- `asserts` в импортах заменён на `with` (в соответствии с синтаксисом атрибутов импорта ECMAScript).
- Директивы `/// <reference no-default-lib />` больше не учитываются при `skipDefaultLibCheck`.
- CLI-сборки не могут принимать пути к файлам, если в текущей директории есть `tsconfig.json`, без явного флага `--ignoreConfig`.

Типы шаблонных литералов теперь сохраняют кодовые точки Unicode

TypeScript 7.0 теперь обрабатывает кодовые точки Unicode более естественно при выводе типов из шаблонных литералов:

```
type HeadTail<S> = S extends `${infer Head}${infer Tail}` ? [Head, Tail] : never

type Result = HeadTail<"😊abc">;
//      ^
// В 7.0: ["😊", "abc"]
// Раньше: ["\ud83d", "\ude00abc"]
```

Объяснить с 

Раньше TypeScript следовал индексации UTF-16 в JavaScript и разбивал "😊" на две половины суррогатной пары. Это было технически согласованно с индексацией в JS, но обычно не то, что имелось в виду, — и могло создавать строковые литеральные типы с непарными суррогатами.

Это ломающее изменение для манипуляций со строками на уровне типов, намеренно моделирующих кодовые единицы UTF-16. На практике новое поведение более интуитивно: вывод типов из шаблонных литералов теперь следует той же логике, что итерация строки через `for...of` или `spread [...str]`, где "😊" считается одной единицей.

Изменения в поддержке JavaScript

При портировании кодовой базы мы также пересмотрели поддержку JavaScript.

TypeScript 7.0 переработал поддержку JS, сделав её более согласованной с анализом TypeScript-файлов. Основные изменения:

- Значения нельзя использовать там, где ожидаются типы — вместо этого пишите `typeof someValue`.

- `@enum` больше не распознаётся специально — используйте `@typedef` на `(typeof YourEnumDeclaration)[keyof typeof YourEnumDeclaration]`.
- Одиночный `?` больше нельзя использовать как тип — используйте `any`.
- `@class` не превращает функцию в конструктор — используйте объявление `class`.
- Постфикс `!` не поддерживается — просто используйте `T`.
- Имена типов должны быть определены внутри тега `@typedef ( /** @typedef {T} TypeAliasName / ), а не рядом с идентификатором ( /* @typedef {T} */ TypeAliasName; )`.
- Синтаксис функций в стиле Closure (например, `function(string): void`) больше не поддерживается — используйте сокращения `TypeScript( (s: string) => void )`.

Также некоторые паттерны JavaScript, вроде псевдонимов `this` и переназначения всего `prototype` функции, больше не обрабатываются специально.

Детальные различия между TypeScript 6.0 и 7.0 фиксируются в файле `CHANGES.md`.

Опыт работы в IDE

Как упоминалось выше, улучшения производительности TypeScript 7.0 касаются не только командной строки, но и IDE.

Для пользователей VS Code есть [специальное расширение](#) — после установки оно автоматически становится основным. Отключить и включить его можно в любой момент через команды «Disable TypeScript 7 Language Server» и «Enable TypeScript 7 Language Server» в командной палитре. В ближайшие недели поддержка TypeScript 7 войдёт в сам VS Code.

Для пользователей Visual Studio последняя версия IDE включает TypeScript 7 автоматически на основе рабочего пространства — ничего дополнительно делать не нужно.

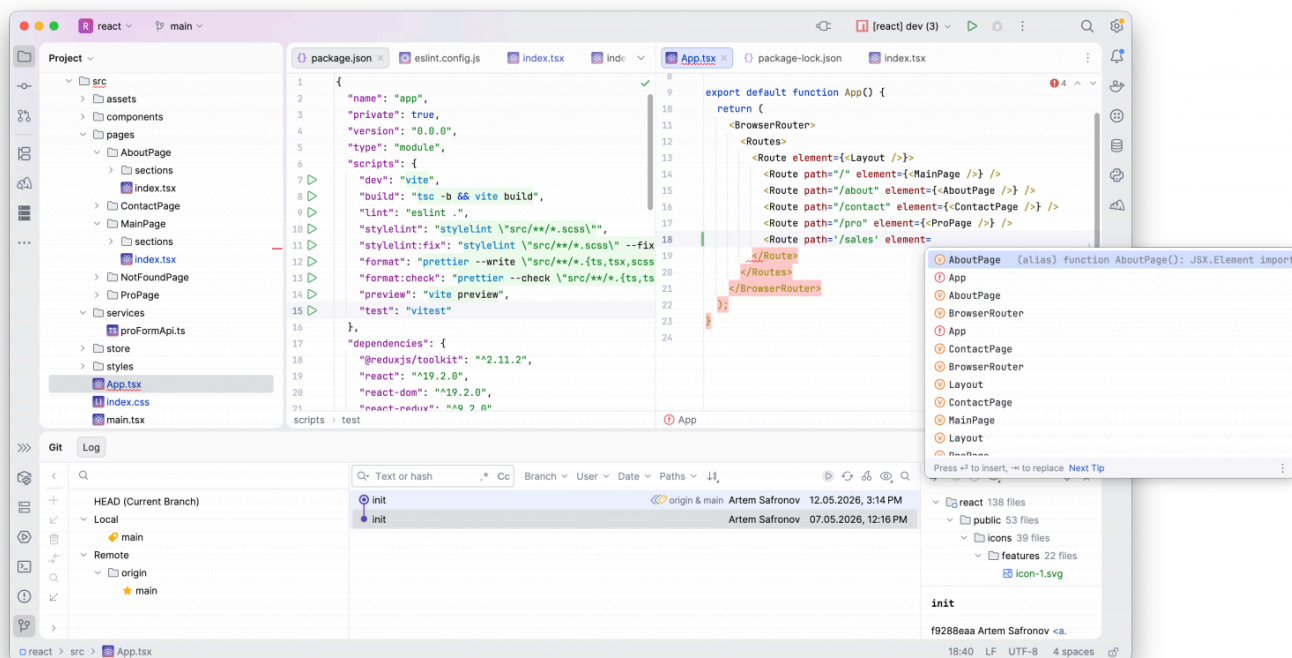
TypeScript 7 отлично работает в любом редакторе. Новая основа построена на LSP и может использовать несколько потоков для обслуживания одновременных запросов максимально быстро.

TypeScript и JavaScript в [OpenIDE](#) предоставляют полноценную поддержку: подсветка, автодополнение, навигация по коду, рефакторинги, диагностика типов через language service из коробки.

Из фреймворков на старте React, Angular и Vue. Для каждого фреймворка есть визард создания проекта, отдельно для Vite и Next.

Из инструментов качества кода ESLint и Stylelint с диагностикой и быстрыми фиксами, которые корректно работают в подмодулях монорепо, и Prettier, читающий конфиг проекта и работающий в связке с ESLint.

Любимый всеми Git-клиент тоже входит в состав OpenIDE.



С момента первого запуска добавлены: автоимпорты, расширяемые подсказки при наведении, подсказки встроенного типа (inlay hints), линзы кода, переход к исходному определению, связанное редактирование и автодополнение тегов JSX и многое другое. Возможности, отсутствовавшие в бета-версии TypeScript 7.0 — семантическая подсветка, «сортировка импортов», «удаление неиспользуемых импортов» — теперь также добавлены.

TypeScript и встроенные языки

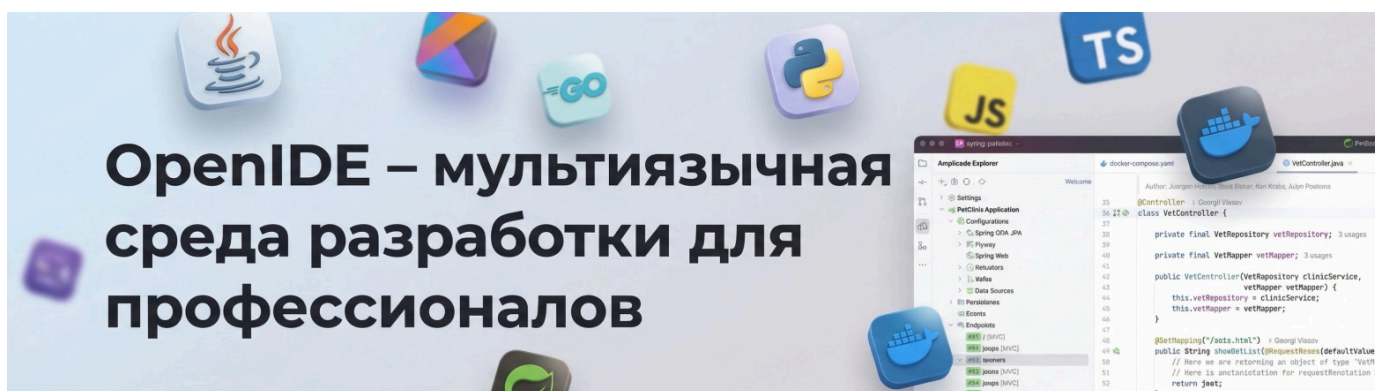
Рабочие процессы, использующие Vue, MDX, Astro, Svelte и другие, пока не смогут использовать TypeScript 7. Аналогично, специализированная проверка типов в шаблонах Angular тоже пока не работает с TypeScript 7. Главная причина — TypeScript 7 ещё не предоставляет стабильного программного API, и инструменты вроде Volar, встраивающие TypeScript в свои компиляторы, могут пока полагаться только на TypeScript 6.0.

Мы ожидаем, что это временная ситуация, и активно работаем с мейнтейнерами этих проектов над её решением.

До тех пор рекомендуем командам использовать TypeScript 7 в сценариях, где не требуются плагины языкового сервера. Проекты на Angular могут комбинировать TypeScript 7 (быстрое обнаружение ошибок через `tsc`) и TypeScript 6.0 (поддержка в редакторе). Проекты на Vue, MDX, Astro, Svelte пока остаются на TypeScript 6.0.

Дорога вперёд

TypeScript 7.0 — важный этап в истории проекта. Этот порт был главным фокусом нашей команды более года. С выходом 7.0 мы возвращаемся к работе над новыми возможностями языка, эргономическими улучшениями, дальнейшим ускорением и реализацией нового API для экосистемы. Ожидаем примерно такой же цикл релизов, как до TypeScript 7.0: новые версии с возможностями каждые 3–4 месяца. TypeScript 7.1 уже на горизонте.



Уже сейчас OpenIDE позволяет разрабатывать проекты на Java, Spring, Python, Go, PHP, JavaScript и TypeScript! А полноценный DB-клиент, поддержка Docker и 300+ плагинов доступны абсолютно бесплатно в маркетплейсе. Пробуйте российскую IDE в деле и подписывайтесь на нас в Telegram или Max, чтобы не пропустить свежие обновления и полезные материалы.

Теги: TypeScript 7, TypeScript Go, нативный TypeScript, ускорение TypeScript, TypeScript компилятор, tsc производительность, TypeScript 7.0 релиз, переход TypeScript 7, TypeScript LSP, TypeScript параллелизация

Хабы: Блог компании Haulmont, Программирование, JavaScript, TypeScript

Редакторский дайджест

Присылаем лучшие статьи раз в месяц

Электронная почта

Подписаться

Оставляя почту, я принимаю Политику конфиденциальности и даю согласие на получение рассылок



64K+

Охват за 30 дней

Haulmont

Сайт Хабр Карьера Telegram ВКонтакте



64K+

20

Охват за 30 дней Карма

Георгий Власов @honest_niceman

Легенда

Подписаться



Комментировать

Публикации

ЛУЧШИЕ ЗА СУТКИ ПОХОЖИЕ



GlinkinIvan
22 часа назад

Хакер в магазине: изучаем поверхность атаки типичного супермаркета

🕒 9 мин 👁 14K

📦 +57

🔖 48

💬 39



medicalofficer1111

23 часа назад

Пить коллаген — это развод?

🕒 7 мин 👁 9.4K

📦 +37

🔖 40

💬 22



arturdumchev

23 часа назад

Калвиаутра путает калвиши: один байт изменил всё

🔗 Средний 🕒 8 мин 👁 11K

Тutorial

📦 +36

🔖 7

💬 7



PatientZero

3 часа назад

Теория мёртвой экономики

🕒 21 мин 👁 6.8K

Перевод

📦 +25

🔖 17

💬 25



nastyakopi

22 часа назад

ИИ-роутер, open-source RAG-платформа, ClickHouse-as-a-service и другие апдейты в продуктах Selectel в июне

🕒 7 мин 👁 11K

Дайджест

📦 +24

🔖 7

💬 3



inetstar
21 час назад

Полный геном за \$200 и его анализ в домашних условиях: Just-DNA-Lite, ИИ и пересборка генома. Часть 3



Средний



29 мин



10K

Тutorial



+20



17



4



SLY_G
19 часов назад

Сколько элементарных частиц существует на самом деле?



Простой



9 мин



10K

Перевод



+19



16



12



xraizor
4 часа назад

Усталость от напарника-машины. Изнанка работы с ИИ-код-агентами



Средний



11 мин



5.4K

Мнение



+16



7



8



AWE64
15 часов назад

Мне надоело писать один и тот же код. Поэтому я сделал Featuregen



Средний



6 мин



13K

Кейс



+16



14



4



inspired_xy
17 часов назад

Как мы съездили в Японию и запустили Пополаму — сервис, который знает кто кому должен

Простой 4 мин 11K

Кейс

Из песочницы

+16

13

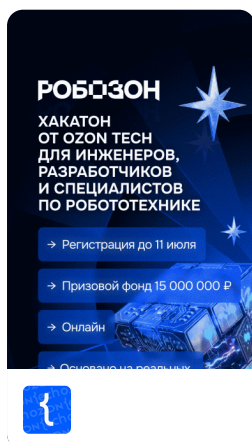
4

Суперсилы и слабости IT-компаний: началось исследование работодателей 2026

Турбо

Показать еще

ИСТОРИИ



Унеси с Робозона свои миллионы



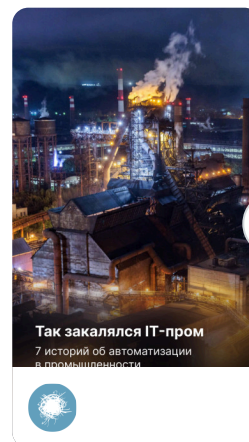
Пришёл, смастерил, рассказал — сезон DIY в разгаре



От лунного овоща до сверхострого чили



Из центра Млечного Пути



Только IT-пром, только хардкор

МИНУТОЧКУ ВНИМАНИЯ



Насколько глубоко ИИ проник в разработку? Опрос



Вы инженер в промышленности? Каким



DPL в Overlay-слое: общение виртуальных машин

статьям вы доверяете?



 [Настройка языка](#)

[Техническая поддержка](#)

© 2006–2026, Habr